

PROPUESTA DE PROYECTO: Sistema de Gestión de Inventario

Grupo: 1

Integrantes: Alasino Benjamin, Calvo Tomas, Contreras Joaquin y Gonzalez Martin.

1. NOMBRE DEL PROYECTO: StockSync

2. PROBLEMA QUE RESUELVE:

StockSync resuelve la pérdida de ingresos y el descontrol operativo causado por la gestión manual o descentralizada de inventarios. El sistema mitiga las mermas mediante trazabilidad y control de accesos estricto, previene quiebres de stock mediante alertas automatizadas de reabastecimiento, y asegura la consistencia de los datos en tiempo real bajo escenarios de uso concurrente."

- **Gestión de Productos Complejos (Catálogo):** Aquí está tu desafío principal. No puedes tener un solo registro llamado "Remera". Debes programar la lógica para crear un "Producto Padre" (SKU Base) que genere automáticamente "Productos Hijo" (Variantes) combinando atributos como Talle (S, M, L) y Color (Rojo, Azul).
- **Kardex / Movimientos de Stock (El Motor):** Un registro inmutable (append-only) de cada unidad que entra o sale. El stock actual no debe ser un número que alguien edite a mano, debe ser la sumatoria calculada de todas las entradas y salidas de ese Kardex.
- **Módulo de Ventas (Egresos concurrentes):** Una interfaz rápida para buscar variantes, validar si hay stock real en milisegundos y registrar la salida sin generar saldos negativos si dos vendedores operan al mismo tiempo.
- **Módulo de Compras (Ingresos):** Para dar de alta mercadería, vincularla a un proveedor y actualizar costos.
- **Seguridad y Auditoría (RBAC):** Roles estrictos. Un vendedor solo puede leer y descontar. Solo el Administrador puede hacer ajustes de stock por mermas o devoluciones.

3. USUARIOS / PÚBLICO OBJETIVO:

Negocios medianos y pequeños (PyMEs) con gestión de stock: comercios minoristas, depósitos, tiendas con múltiples empleados. Roles contemplados: administrador (acceso total), operario/empleado (carga y consulta).

4. ALCANCE FUNCIONAL - al menos 5 requerimientos:

- **REQ-F01:** El sistema debe permitir la creación de productos "Padre" y la generación automatizada de SKUs "Hijo" mediante una matriz de variantes parametrizable (ej. Talle, Color, Temporada).

- **REQ-F02:** El sistema debe registrar toda entrada, salida o ajuste mediante un Kardex transaccional inmutable, almacenando timestamp, ID de usuario, tipo de movimiento y cantidad afectada.
- **REQ-F03:** El sistema debe procesar solicitudes de venta descontando el stock, validando la disponibilidad exacta de la variante antes del commit transaccional para impedir saldos en negativo.
- **REQ-F04:** El sistema debe notificar alertas de "Punto de Reorden" cuando el stock calculado de una variante caiga por debajo del umbral de seguridad definido por el administrador.
- **REQ-F05:** El sistema debe aplicar un modelo RBAC (Control de Acceso Basado en Roles), bloqueando a nivel de API las operaciones de "Ajuste de Inventario" y "Eliminación Lógica" para cualquier usuario que no posea el rol de Administrador.

5. ALCANCE NO FUNCIONAL - al menos 2 requerimientos:

- **REQ-NF01 (Integridad):** El sistema debe garantizar un nivel de aislamiento transaccional *Read Committed* en la base de datos para prevenir lecturas sucias y asegurar la consistencia del inventario bajo alta concurrencia.
- **REQ-NF02 (Rendimiento):** Las consultas de validación de disponibilidad de stock (búsqueda de un SKU hijo) deben resolverse en un tiempo de respuesta menor a 300ms en el percentil 95.

6. STACK TECNOLÓGICO:

- **Lenguaje:** Python (Backend) / TypeScript (Frontend).
- **Framework:** React, Express, Node.JS
- **Base de datos:** PostgreSQL
- **Otras herramientas:** Git/GitHub (aplicando flujo de trabajo GitFlow y convenciones estrictas de *commits*), Docker (para estandarizar el entorno de desarrollo y despliegue) y Postman para la documentación de la API.

7. REPOSITORIO (URL pública en GitHub/GitLab) :

<https://github.com/benjaalasino/Ing.-de-Software-II>

8. RIESGOS PRINCIPALES Y CÓMO LOS MITIGARÁN (2-3 ítems) :

- **Riesgo 1:** Explosión combinatoria de variantes y degradación de rendimiento. Un producto con 5 colores y 5 talles genera 25 SKUs distintos. Si el modelo de base de datos es rígido, las consultas colapsarán.
→ Mitigación: Se diseñará un modelo de datos basado en el patrón EAV (Entity-Attribute-Value) o utilizando campos JSON para los atributos dinámicos, aplicando índices optimizados para garantizar búsquedas en tiempo real.
- **Riesgo 2:** Condición de carrera (Race Conditions) en ventas concurrentes. Dos usuarios intentando vender la última unidad de un talle específico simultáneamente podrían generar stock negativo.

→ Mitigación: Se implementarán transacciones ACID utilizando bloqueos optimistas (Optimistic Locking) a nivel de base de datos para asegurar que la lectura y actualización de disponibilidad sea atómica.

- **Riesgo 3:** Pérdida de integridad y "robo hormiga" por manipulaciones directas.
→ Mitigación: El stock se calculará en base a un Kardex inmutable (Log de transacciones). No existirán operaciones UPDATE o DELETE sobre la tabla de cantidades; todo ajuste será un nuevo registro de compensación auditado y ligado al ID del usuario responsable.